

README – Práctica No. 2: API de Procesos (Shell wish)

1. Integrantes

- **Nombre:** Cristian Flórez
- **Correo:** cristian.florez@udea.edu.co
- **Documento:** 1040183014

2. Descripción de la solución

En la presente práctica se desarrolló un intérprete de comandos (shell) denominado wish, el cual replica el comportamiento básico de un shell en sistemas Unix/Linux. La implementación se realizó en lenguaje C, siguiendo las especificaciones establecidas en la guía del laboratorio.

El funcionamiento del shell se basa en un ciclo iterativo que, en modo interactivo, presenta un prompt al usuario (wish>), recibe una línea de entrada, la analiza sintácticamente y ejecuta el comando correspondiente. Este proceso se repite de forma continua hasta que el usuario ingresa el comando de salida.

El sistema desarrollado soporta dos modos de operación:

- **Modo interactivo:** el usuario ingresa comandos directamente desde la terminal.
- **Modo batch:** el shell recibe un archivo como argumento y ejecuta cada línea contenida en este.

El diseño del shell contempla la diferenciación entre comandos internos (built-in) y comandos externos. Los comandos internos (exit, chd, route) son ejecutados directamente por el proceso principal, mientras que los comandos externos son ejecutados mediante la creación de procesos hijos utilizando fork() y execv().

Adicionalmente, se implementaron funcionalidades avanzadas como:

- Redirección de salida estándar y de error mediante el operador >
- Ejecución de comandos en paralelo mediante el operador &
- Gestión dinámica del search path para la localización de ejecutables

3. Documentación de funciones

print_error()

Imprime el mensaje de error estándar en la salida de error (stderr) utilizando la llamada al sistema write(). Este mensaje es utilizado de forma uniforme para todos los errores, tal como lo exige la especificación.

init_path()

Inicializa el arreglo global que almacena el search path, asignando como valor inicial el directorio /bin. Se reserva memoria dinámicamente y se establece el contador de rutas.

free_path()

Libera la memoria previamente asignada al search path. Esta función es utilizada principalmente cuando el comando route modifica la lista de directorios.

find_executable(char *cmd)

Busca un ejecutable dentro de los directorios definidos en el search path. Para cada ruta, construye la dirección completa del ejecutable y utiliza access () con el flag X_OK para verificar su existencia y permisos de ejecución. Retorna la ruta completa si se encuentra, o NULL en caso contrario.

builtin_exit(char **args, int argc)

Implementa el comando interno exit. Verifica que no se proporcionen argumentos adicionales y finaliza la ejecución del shell mediante exit (0).

builtin_chd(char **args, int argc)

Implementa el comando interno chd, el cual permite cambiar el directorio de trabajo del proceso utilizando la llamada al sistema chdir (). Se valida que se reciba exactamente un argumento y se maneja el error en caso de que el directorio no exista o no sea accesible.

builtin_route(char **args, int argc)

Permite redefinir completamente el search path. Recibe como argumentos los nuevos directorios y reemplaza el path anterior. Si no se proporcionan argumentos, el path queda vacío, lo que impide la ejecución de comandos externos.

parse_single_command(char *line)

Analiza una cadena de entrada correspondiente a un único comando. Se encarga de:

- Separar los tokens utilizando `strsep()`
- Identificar el operador de redirección (`>`)
- Validar errores de sintaxis relacionados con redirección
- Construir una estructura `Command` con los argumentos y metadatos necesarios

free_command(Command *cmd)

Libera la memoria asociada a una estructura `Command`, incluyendo sus argumentos y el nombre del archivo de redirección en caso de existir.

execute_command(Command *cmd, int wait_child)

Ejecuta un comando individual. Si se trata de un comando interno, se invoca directamente la función correspondiente. Si es un comando externo, se utiliza `fork()` para crear un proceso hijo, el cual ejecuta el programa mediante `execv()`.

En caso de redirección, el proceso hijo redirige los descriptores de archivo estándar (`stdout` y `stderr`) hacia el archivo especificado utilizando `dup2()`.

El proceso padre puede esperar o no la finalización del hijo dependiendo del parámetro `wait_child`.

run_line(char *line)

Procesa una línea completa de entrada. Divide la línea en múltiples comandos utilizando el operador `&` para ejecución en paralelo. Cada segmento es parseado y ejecutado como un comando independiente. Si hay múltiples comandos, estos se ejecutan concurrentemente y posteriormente el proceso padre espera la finalización de todos.

main(int argc, char *argv[])

Función principal del programa. Determina el modo de operación (interactivo o batch), inicializa el search path y gestiona el ciclo principal del shell. Utiliza `getline()` para leer la entrada y finaliza correctamente al detectar el fin de archivo (EOF).

4. Problemas presentados y soluciones

Durante el desarrollo de la práctica se presentaron dificultades relacionadas principalmente con el entorno de ejecución y compilación del programa, dado que la implementación depende de llamadas al sistema propias de sistemas operativos tipo Unix.

Inicialmente, se intentó compilar el programa en un entorno Windows utilizando la terminal integrada del editor, lo cual generó errores como:

```
fatal error: sys/wait.h: No such file or directory
```

Este problema se debe a que funciones como `fork()`, `execv()` y `wait()` pertenecen al estándar POSIX y no están disponibles en entornos Windows nativos. Para solucionar esta limitación, se optó por instalar Windows Subsystem for Linux (WSL), específicamente en su versión 1, debido a restricciones de virtualización en el equipo.

Una vez instalado WSL, se procedió a configurar el entorno Linux e instalar las herramientas necesarias mediante los comandos:

```
sudo apt update  
sudo apt install build-essential
```

Posteriormente, se compiló el programa con:

```
gcc -o wish wish.c
```

Durante las pruebas iniciales se presentó el siguiente error al intentar ejecutar el binario desde una terminal de Windows:

```
cannot execute binary file: Exec format error
```

Este error se produjo debido a que el ejecutable generado en WSL corresponde a un formato binario de Linux, el cual no puede ser ejecutado desde entornos como Git Bash. La solución consistió en ejecutar el programa directamente dentro de la terminal de Ubuntu (WSL), donde sí existe compatibilidad con este tipo de binarios:

```
./wish
```

Otro aspecto que generó discusión fue la nomenclatura del comando `chd`, el cual es mencionado en la guía. Dado que en sistemas Unix el comando estándar es `cd`, se identificó una posible inconsistencia en la documentación. No obstante, se decidió implementar el comando `chd` conforme a lo indicado en la guía, utilizando internamente la llamada al sistema `chdir()`.

5. Pruebas realizadas

Se realizaron diversas pruebas para validar el correcto funcionamiento del shell en todos los escenarios requeridos.

Para verificar la ejecución de comandos externos:

```
wish> ls  
wish> pwd
```

Para validar el manejo de errores:

```
wish> comando_invalido
```

En este caso, el sistema responde correctamente con el mensaje de error estándar.

Para comprobar el funcionamiento del cambio de directorio:

```
wish> chd ..
```

Para validar la redirección de salida estándar y de error:

```
wish> ls > salida.txt
```

Se verificó que tanto la salida normal como los errores se almacenan en el archivo especificado.

Para evaluar la ejecución de comandos en paralelo:

```
wish> ls & pwd & echo prueba
```

Se observó que los comandos se ejecutan de manera concurrente y el shell espera la finalización de todos los procesos antes de continuar.

Finalmente, se probó el modo batch mediante un archivo de entrada:

```
./wish archivo.txt
```

Confirmando que los comandos se ejecutan secuencialmente sin mostrar el prompt.

6. Video de sustentación

Enlace al video de sustentación: <https://youtu.be/Dt-8XR3VLIs>

7. Manifiesto de transparencia

Durante el desarrollo de esta práctica utilicé herramientas de inteligencia artificial como apoyo en la comprensión de conceptos relacionados con la creación y gestión de procesos en sistemas operativos, así como en la resolución de errores de compilación y configuración del entorno.

El uso de estas herramientas se centró en:

- Explicación de llamadas al sistema como `fork()`, `execv()` y `wait()`
- Diagnóstico de errores en el entorno de desarrollo
- Asistencia en la redacción técnica del informe

No obstante, la implementación del programa, la integración de funcionalidades, la realización de pruebas y la validación del comportamiento del sistema fueron producción del mía, garantizando la apropiación de los conocimientos abordados en la práctica.

8. Conclusión

El desarrollo de esta práctica permitió comprender en profundidad el funcionamiento de un intérprete de comandos y el manejo de procesos en sistemas operativos tipo Unix. Se logró implementar un shell funcional que cumple con los requerimientos establecidos, incluyendo ejecución de comandos, manejo de rutas, redirección de salida y ejecución en paralelo, consolidando así los conocimientos teóricos mediante su aplicación práctica.